so-02.1-scheduling.pdf

Scheduler, processi e thread

Un sistema operativo è un gestore di risorse
processore, memoria principale e secondaria, dispositivi

Per svolgere i suoi compiti, un sistema operativo ha bisogno di strutture dati per mantenere informazioni sulle risorse gestite

Queste strutture dati comprendono:
tabelle di memoria
tabelle di I/O
tabelle del file system
tabelle dei processi $\longrightarrow$ Argomento di questo modulo

Tabelle per la gestione della memoria

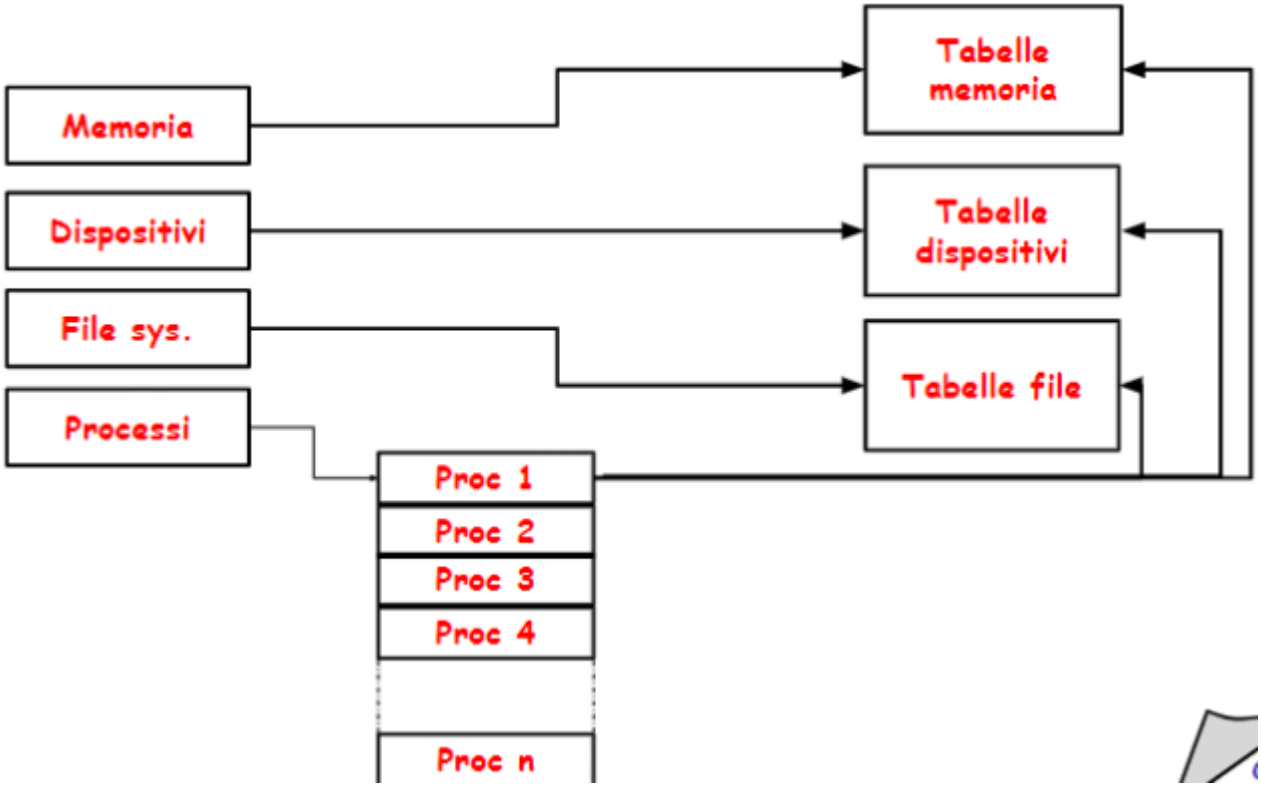
- allocazione memoria per il sistema operativo
- allocazione memoria principale e secondaria per i processi
- informazioni per i meccanismi di protezione

Tabelle per la gestione dell'I/O

- informazioni sullo stato di assegnazione dei dispositivi utilizzati dalla macchina gestione di code di richieste

Tabelle per la gestione del file system

- elenco dei dispositivi utilizzati per mantenere il file system
- elenco dei file aperti e loro stato

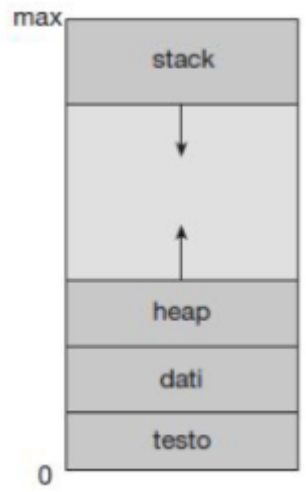


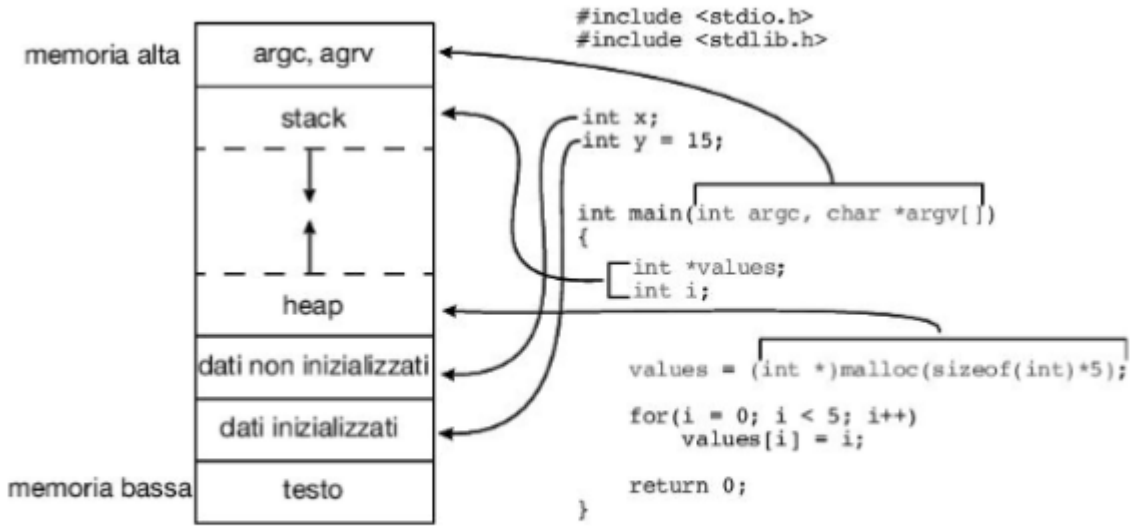
Descrittori dei processi

Qual é la manifestazione fisica di un processo?

1. il codice da eseguire (segmento codice)
2. i dati su cui operare (segmenti dati)
3. uno stack di lavoro per la gestione di chiamate di funzione, passaggio di parametri e variabili locali
4. un insieme di attributi contenenti tutte le informazioni necessarie per la gestione del processo stesso
   Questo insieme di attributi prende il nome di
   descrittore del processo (PROCESS CONTROL BLOCK , PCB)

PROCESSO IN MEMORIA





vedi **esempio_allocazione.c**

Tabella per la gestione dei processi

→contiene i descrittori dei processi e ogni processo ha un descrittore associato

E' possibile suddividere le informazioni contenute nel descrittore in tre aree:

→informazioni di identificazione di processo

→informazioni di stato del processo

→informazioni di controllo del processo

Informazioni di identificazione di un processo

identificatore di processo (process id, o pid)

→*può essere semplicemente un indice all'interno di una tabella di processi*

→*può essere un numero progressivo*; in caso, è necessario un mapping tra pid e posizione del relativo descrittore →molte altre tabelle del SO utilizzano il process id per identificare un elemento della tabella dei processi

identificatori di altri processi logicamente collegati al processo → ad esempio, **pid** del

processo padre
id dell'utente che ha richiesto l'esecuzione del processo

Informazioni di stato del processo

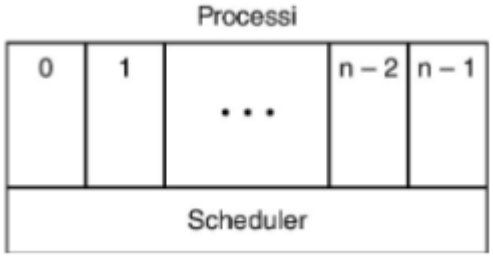
- registri generali del processore
- registri speciali, come il program counter e i registri di stato

Informazioni di controllo del processo
Informazioni di scheduling ⟶stato del processo : in esecuzione, pronto, in attesa
→ *informazioni particolari necessarie dal particolare algoritmo di scheduling utilizzato*
⟶ priorità, puntatori per la gestione delle code
→*identificatore dell'evento per cui il processo è in attesa*
informazioni di gestione della memoria
→ *valori dei registri base e limite dei segmenti utilizzati, puntatori alle tabelle delle pagine, etc.*
informazioni di accounting
tempo di esecuzione del processo
tempo trascorso dall'attivazione di un processo
informazioni relative alle risorse
→ *risorse controllate dal processo, come file aperti, device allocati al processo*
informazioni per **interprocess communication (IPC)**
stato di segnali, semafori, etc.

SCHEDULER

E' la componente più importante del kernel
Gestisce l'avvicendamento dei processi :
⟹*decide quale processo deve essere in esecuzione ad ogni istante*
⟹interviene quando viene richiesta un'operazione di I/O e quando un'operazione di I/O termina, ma anche periodicamente

NB
⟶ Il termine "**SCHEDULER**" viene utilizzato anche in altri ambiti con il significato di "**GESTORE DELL'AVVICENDAMENTO DEL CONTROLLO**".
⟶possiamo quindi fare riferimento allo "scheduler del disco", e in generale allo "scheduler del dispositivo X".

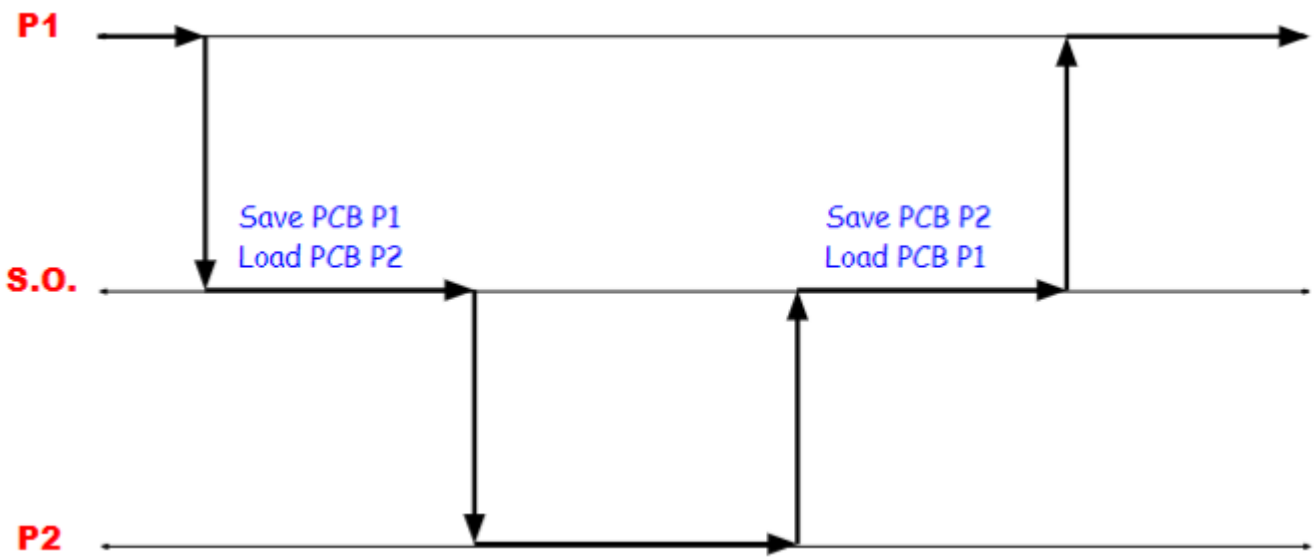


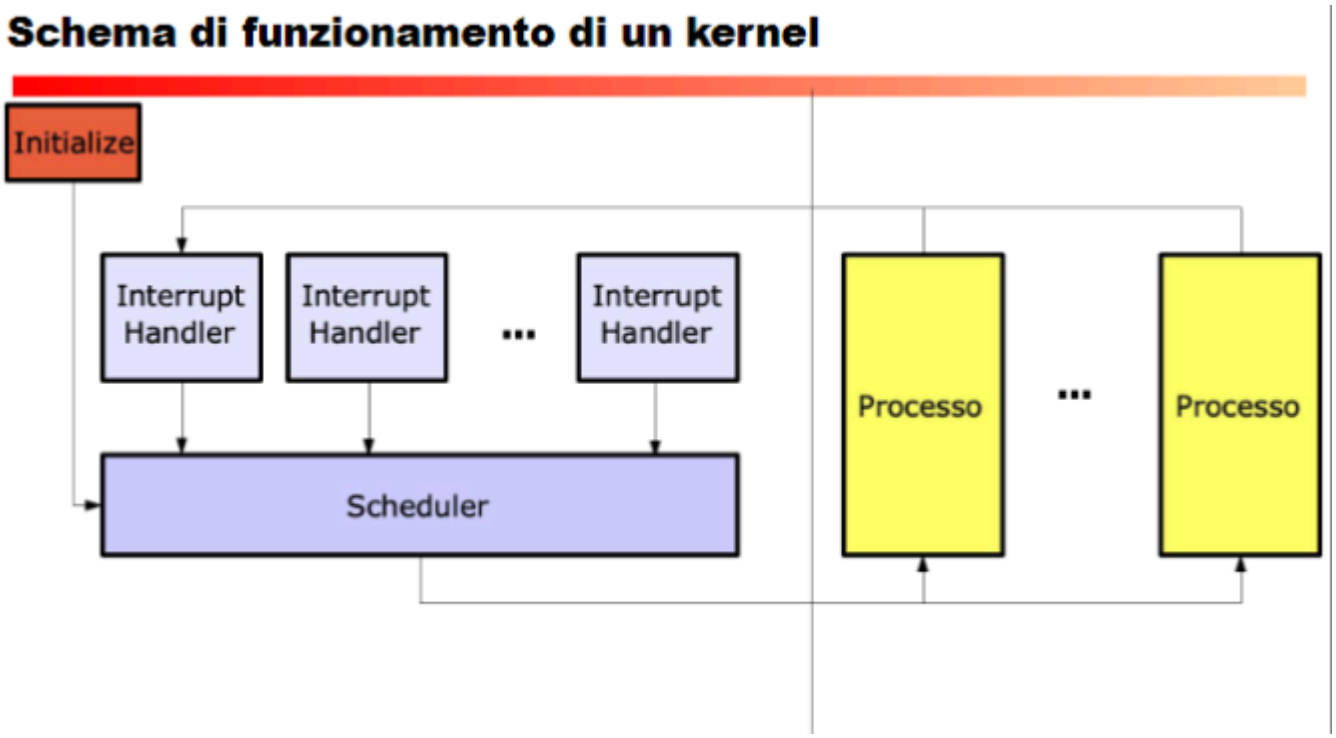
Schedule è la sequenza temporale di assegnazioni delle risorse da gestire ai richiedenti
Scheduling è l'azione di calcolare uno schedule
Scheduler è la componente software che calcola lo schedule

Mode switching e context switching

Tutte le volte che avviene un interrupt (software o hardware) il processore è soggetto ad un MODE SWITCHING
modalità utente → modalità supervisore AVVIENE QUINDI UN CAMBIO DI "SICUREZZA"
Durante la gestione dell'interrupt
→ vengono intraprese le opportune azioni per gestire l'evento
→ viene chiamato lo scheduler: *se lo scheduler decide di eseguire un altro processo*, il sistema è soggetto ad un context switching QUINDI AZIONI I/O → det. Mode switch → se necessario Scheduler esegue un altro processo → definito un context switch

Operazioni durante un context switching Process control block
→lo stato del processo attuale viene salvato nel **PCB** corrispondente
→lo stato del processo selezionato per l'esecuzione viene caricato dal PCB nel processore





Esempio →Vedi ***simulazione_interrupt.c***
Il programma inizia in "modalità utente"
Usa *signal()* per impostare un gestore per l'interrupt SIGINT (Ctrl+C)
In un ciclo infinito, simula il lavoro stampando 'U' (utente) o 'K' (kernel)
Quando l'utente preme Ctrl+C, viene chiamato interrupt_handler():
■ Entra in "modalità kernel"
■ Simula il lavoro di gestione dell'interrupt
■ Torna in "modalità utente"

---

approfondimento codice
Vedi ***simulazione_interrupt.c***
→ variable in_kernel_mode è dichiarata volatile
→La ragione è legata al modo in cui il compilatore ottimizza il codice e come i segnali interagiscono con il programma

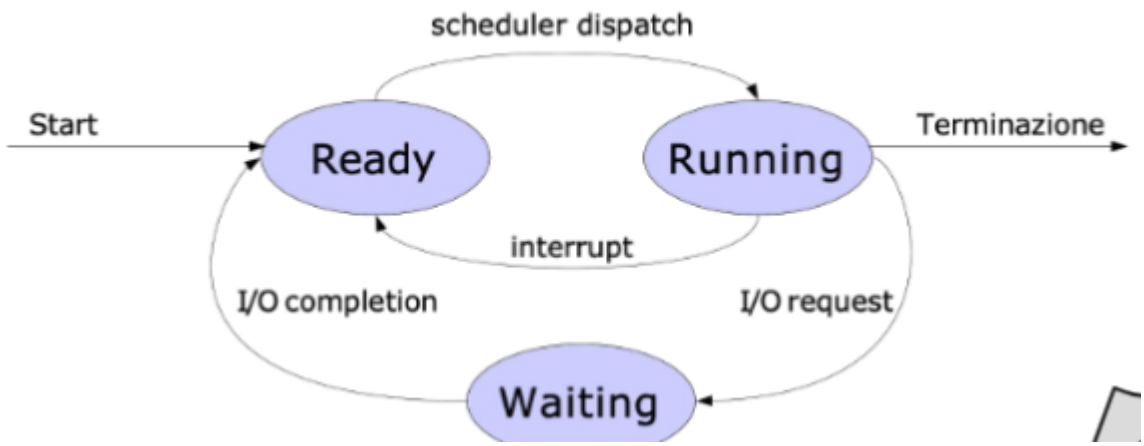
■ Ottimizzazioni del compilatore: Senza `volatile`, il compilatore potrebbe ottimizzare il codice assumendo che `in_kernel_mode` non cambi (no assegnamenti nel loop)
Evitare il caching in registri: **Senza** `volatile`, il compilatore potrebbe decidere di caricare il valore di `in_kernel_mode` in un registro all'inizio del ciclo e non ricaricarlo ad ogni iterazione, perdendo così eventuali modifiche fatte dal gestore di segnali.

■ Modifiche esterne al flusso normale: `in_kernel_mode` viene modificata all'interno del gestore di segnali (`interrupt_handler`), che è una routine esterna al flusso di esecuzione normale del programma

---

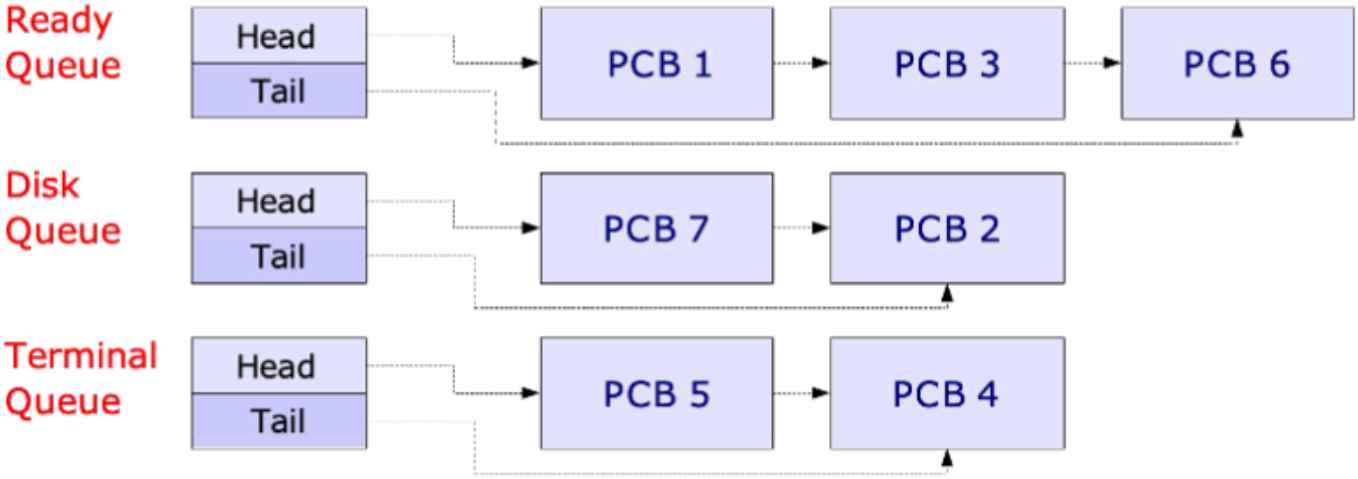
Vita di un processo

Stati dei processi:
Running: il processo è in esecuzione
Waiting: il processo è in attesa di qualche evento esterno (e.g., completamento operazione di I/O); non può essere eseguito
Ready: il processo può essere eseguito, ma attualmente il processore è impegnato in altre attività

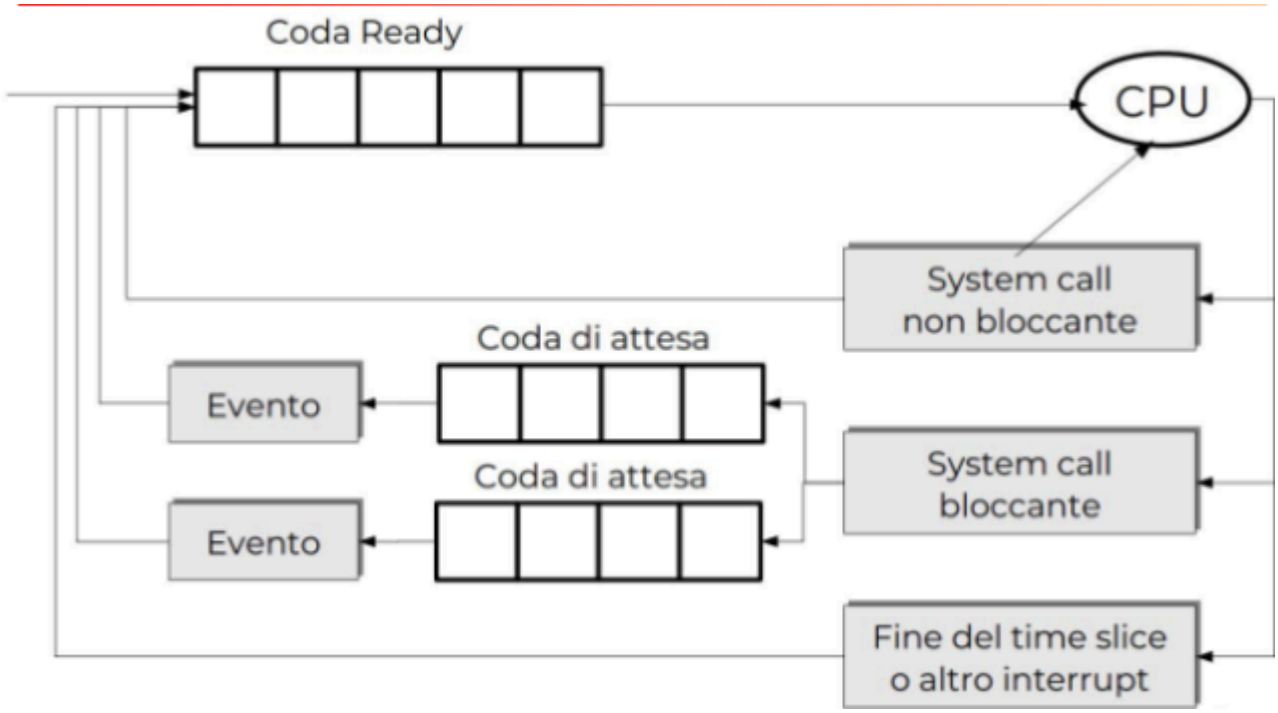


Code di processi

Tutte le volte che un processo entra nel sistema, viene posto in una delle code gestite dallo scheduler



Vita di un processo nello scheduler

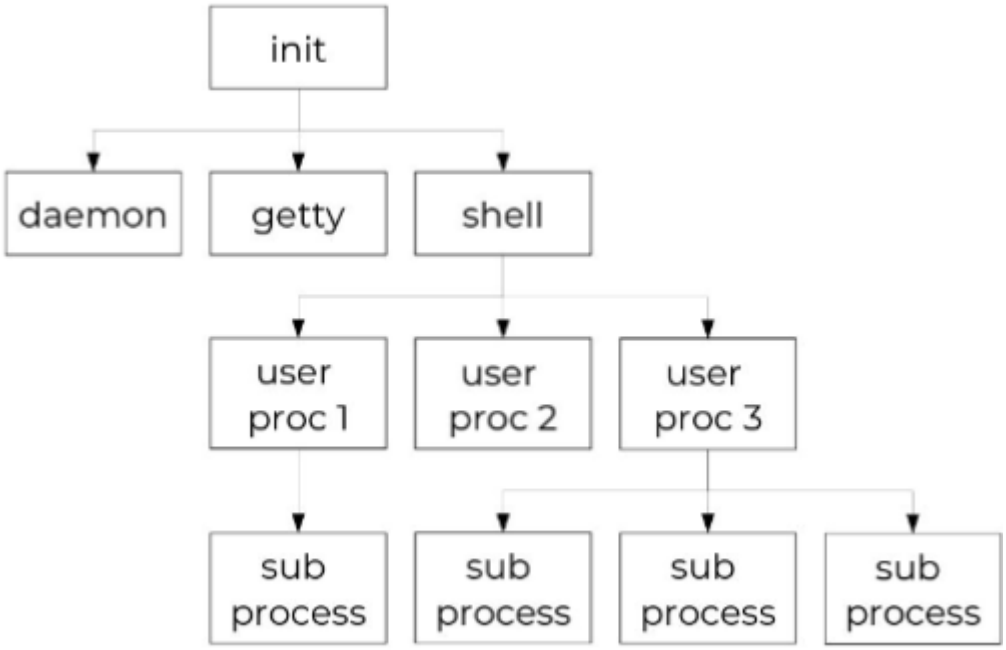


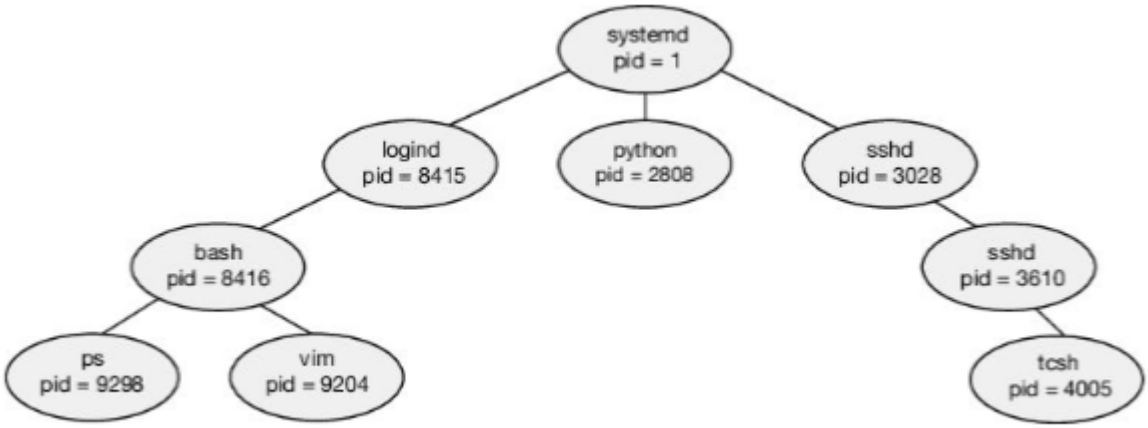
Gerarchia di processi

Nella maggior parte dei sistemi operativi i processi sono organizzati in forma gerarchica ⟶ quando un processo crea un nuovo processo, il processo creante viene detto padre e il creato figlio ⟶ si viene così a creare un albero di processi

Motivazioni : semplificazione del procedimento di creazione di processi ⇒non occorre specificare esplicitamente tutti i parametri e le caratteristiche
→ *ciò che non viene specificato, viene ereditato dal padre*

esempio unix di gerarchia dei processi





Linux: comando ps

Il comando `ps` in Linux è uno strumento utilizzato per visualizzare i processi in esecuzione nel sistema.
Fornisce informazioni dettagliate su ciascun processo, come l'ID del processo (PID), l'utente che lo esegue, la percentuale di CPU e di memoria utilizzate, il tempo di esecuzione e altro.
Di seguito sono illustrati gli utilizzi e le opzioni più comuni di questo comando.

`ps` ⟹ Eseguito senza opzioni, questo comando mostra i processi in esecuzione nel terminale corrente.

1. **Elencare tutti i processi nel sistema**:
   `ps aux`
   - **a**: Mostra i processi di tutti gli utenti.
   - **u**: Mostra informazioni dettagliate, come l'utente, la CPU e la memoria utilizzate.

    - **x**: Mostra i processi senza terminale associato (come quelli in background o di sistema).
- **Visualizzare i processi in formato ad albero**:
  `ps aux --forest`
    - `--forest`: Organizza i processi in un formato ad albero per vedere più facilmente le relazioni di parentela (processo padre e figlio).
- **Visualizzare i processi di un utente specifico**:
  `ps -u nome_utente`
    - `-u`: Permette di filtrare i processi per uno specifico utente.
- **Visualizzare i dettagli di un processo specifico tramite PID**:
  `ps -p <PID>`
    - `-p`: Permette di selezionare un processo specifico tramite il suo ID.

Esempi comuni

- **Visualizzare tutti i processi con informazioni dettagliate**:
  `ps aux`
- **Visualizzare solo i processi dell'utente attuale**:
  `ps -u $(whoami)`
- **Visualizzare tutti i processi di sistema in ordine gerarchico**:
  `ps -ef --forest`
  - **-e**: Mostra tutti i processi di sistema.
  - **-f**: Fornisce un formato di visualizzazione completo.
  Il comando `ps` è spesso usato insieme ad altri comandi come `grep` per filtrare specifici processi. Ad esempio:
  `ps aux | grep nome_programma` ⇒uesto comando mostra solo i processi relativi a "nome_programma".

Il comando `ps -el` in Linux visualizza una lista dettagliata di tutti i processi in esecuzione nel sistema, con informazioni estese grazie all'opzione `-l` (che indica "long format").

```
`ps -el`
```

Linux: comando top
Il comando top fornisce una panoramica in tempo reale delle risorse di sistema e dei processi in esecuzione
Lista dei processi:

- PID: ID del processo.
- USER: Utente proprietario del processo.

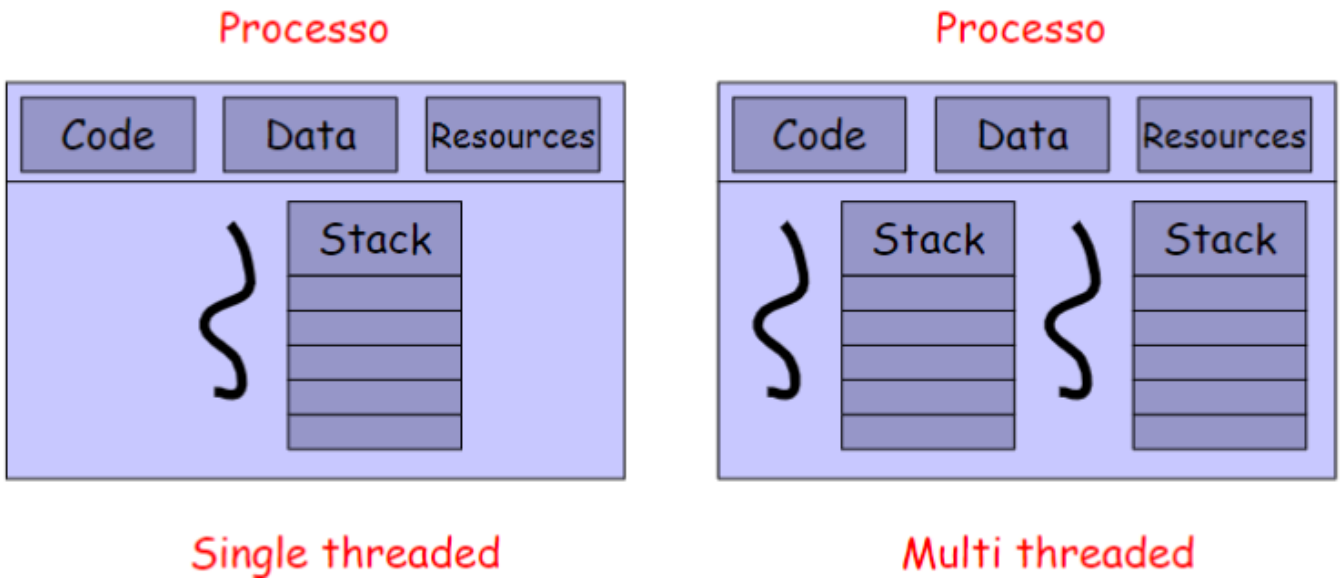
- PR: Priorità del processo.
- NI: Valore nice del processo.
- VIRT: Memoria virtuale utilizzata dal processo.
- RES: Memoria fisica utilizzata dal processo.
- SHR: Memoria condivisa utilizzata dal processo.
- S: Stato del processo (es. S=sleep, R=running, Z=zombie).
- %CPU: Percentuale di utilizzo della CPU da parte del processo.
- %MEM: Percentuale di utilizzo della memoria da parte del processo.

Processi e Thread

La nozione di processo discussa in precedenza assume che ogni processo abbia una singola "linea di controllo"
→ per ogni processo, viene eseguite una singola sequenza di istruzioni
→ un singolo processo non può eseguire due differenti attività contemporaneamente
Esempi:
scaricamento di due differenti pagine in un web browser.
inserimento di nuovo testo in un **wordprocessor** mentre viene eseguito il correttore ortografico.

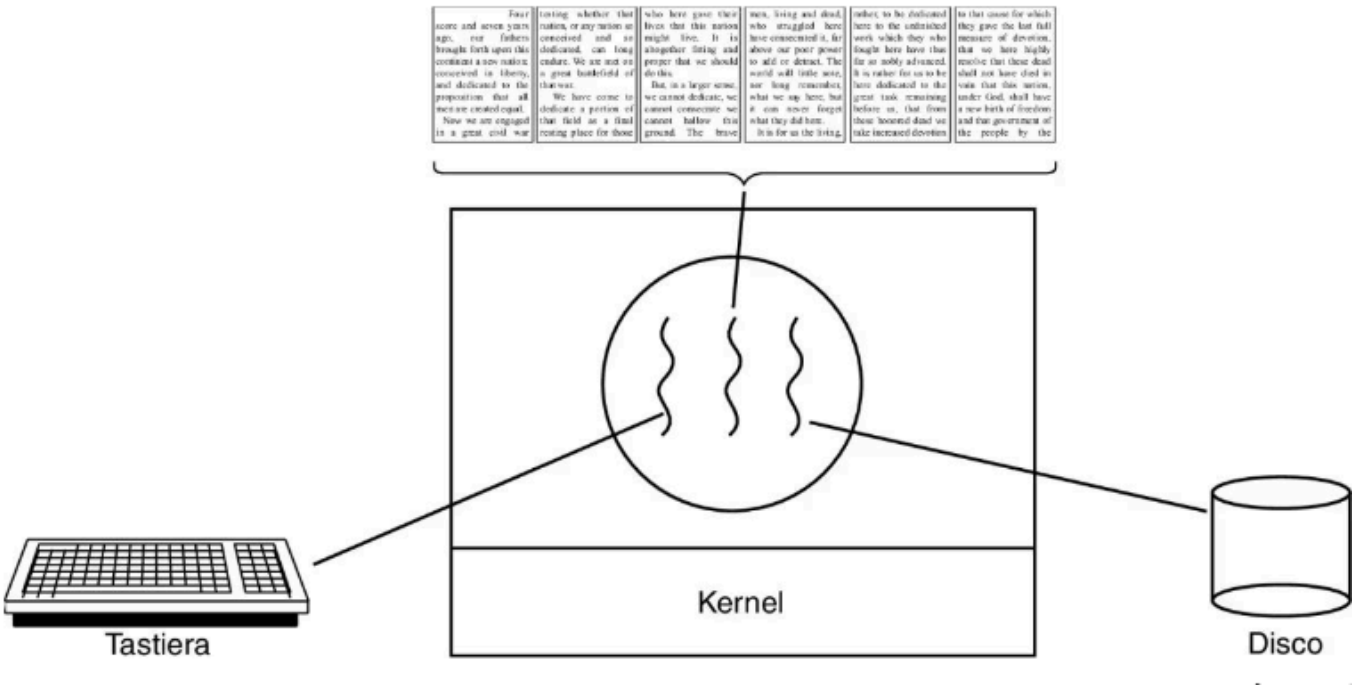
Tutti i sistemi operativi moderni supportano l'esistenza di processi multithreaded
in un processo multithreaded esistono molte "linee di controllo", ognuna delle quali può eseguire un diverso insieme di istruzioni
Esempi:
Associando un thread ad ogni finestra aperta in un web browser, è possibile scaricare i dati in modo indipendente

Un thread è l'unità base di utilizzazione della CPU
Ogni thread possiede:
la propria copia dello stato del processore
il proprio program counter
uno stack separato
I thread appartenenti allo stesso processo condividono:
il codice ; i dati ; le risorse di I/O



Benefici dei thread

Condivisione di risorse i thread condividono lo spazio di memoria e le risorse allocate degli altri thread dello stesso processo
→condividere informazioni tra thread logicamente correlati rende più semplice l'implementazione di certe applicazioni
Esempio: web browser: condivisione dei parametri di configurazione fra i vari thread
UN ELABORATORE DI TESTI CON 3 THREAD



un thread Interagisce con l'utente (legge tastiera)
un altro gestisce l'impaginazione del testo in background
l'ultimo salva in automatico (background) su file

Economia: allocare memoria e risorse per creare nuovi processi è costoso → fare context switching fra diversi processi è costoso
Gestire i thread è in generale più economico, quindi preferibile creare thread all'interno di un processo è meno costoso → fare context switching fra thread è meno costoso
Esempio:
creare un thread in Solaris richiede 1/30 del tempo richiesto per creare un nuovo processo

Processi vs Thread

Thread = processi "lightweight"
utilizzare i thread al posto dei processi rende l'implementazione più efficiente
in ogni caso, abbiamo bisogno di processi distinti per applicazioni differenti

Multithreading: implementazione

Un sistema operativo può implementare i thread in due modi:
User thread (A livello utente)
Kernel thread (A livello kernel)

User Thread (Thread a livello utente)

I **User Thread** sono gestiti completamente dalla **libreria di thread** a livello dell'applicazione, senza coinvolgere direttamente il kernel.
Questi thread esistono solo nello spazio utente e sono invisibili al kernel, il quale vede l'intero processo come una singola entità.

Caratteristiche principali:

- **Gestione a livello utente**: Il sistema operativo non ha alcuna conoscenza diretta di questi thread; tutto viene gestito dal codice dell'applicazione stessa (solitamente tramite una libreria di gestione dei thread).
- **Velocità e Efficienza**: Creare, distruggere e sincronizzare user thread è più veloce e richiede meno risorse, poiché non implica chiamate di sistema né coinvolge il kernel.
- **Limitazioni di concorrenza**: Poiché i user thread non sono riconosciuti dal kernel, solo un thread alla volta può eseguire codice, anche in un ambiente multi-core. Questo limita la capacità di eseguire user thread in parallelo su più CPU.
- **Blocco**: Se un user thread entra in uno stato di blocco (come durante l'accesso a I/O o una risorsa non disponibile), l'intero processo si blocca. Questo è dovuto al fatto che il kernel vede solo un singolo processo e non i singoli thread all'interno di esso.

Vantaggi degli User Thread:

- **Minore overhead**: Operazioni di creazione e gestione dei thread sono leggere e veloci, poiché non coinvolgono il kernel.
- **Portabilità**: Poiché il kernel non è coinvolto, è più facile rendere portabili questi thread tra diversi sistemi operativi.

Svantaggi degli User Thread:

- **Assenza di parallelismo**: Non possono utilizzare appieno il parallelismo su sistemi multi-core, poiché solo un thread può essere attivo alla volta.
- **Blocco del processo**: Se un thread si blocca, l'intero processo può bloccarsi.

Kernel thread

I kernel thread vengono supportati direttamente dal sistema operativo la creazione, lo scheduling e la gestione dei thread sono implementati a livello kernel.
Vantaggi:
poiché è il kernel a gestire lo scheduling dei thread, se un thread esegue una operazione di I/O, il kernel può selezionare un altro thread in attesa di essere eseguito.
Svantaggi:
l'implementazione risultante è più lenta, perché richiede un passaggio da livello utente a livello supervisore.

Kernel Thread (Thread a livello kernel)

I **Kernel Thread** sono gestiti direttamente dal **kernel del sistema operativo**. In questo caso, il kernel ha piena conoscenza e controllo su ciascun thread, e ogni thread è trattato come una singola unità di schedulazione.

Caratteristiche principali:

- **Gestione a livello kernel**: Il kernel crea, gestisce e distrugge i thread, occupandosi anche della loro sincronizzazione e schedulazione.
- **Parallelismo**: I kernel thread possono essere distribuiti tra diversi processori in un sistema multi-core, permettendo così di eseguire thread in parallelo e ottenere un vero parallelismo.
- **Blocco parziale**: Poiché il kernel gestisce ogni thread individualmente, il blocco di un singolo thread non blocca necessariamente l'intero processo. Questo aumenta l'efficienza e riduce i tempi di attesa in caso di operazioni bloccanti.

Vantaggi dei Kernel Thread:

- **Parallelismo reale**: Permettono l'esecuzione parallela su sistemi multi-core, sfruttando appieno la capacità di calcolo.

- **Isolamento di blocco**: Il blocco di un thread non influisce sugli altri thread nello stesso processo.

Svantaggi dei Kernel Thread:

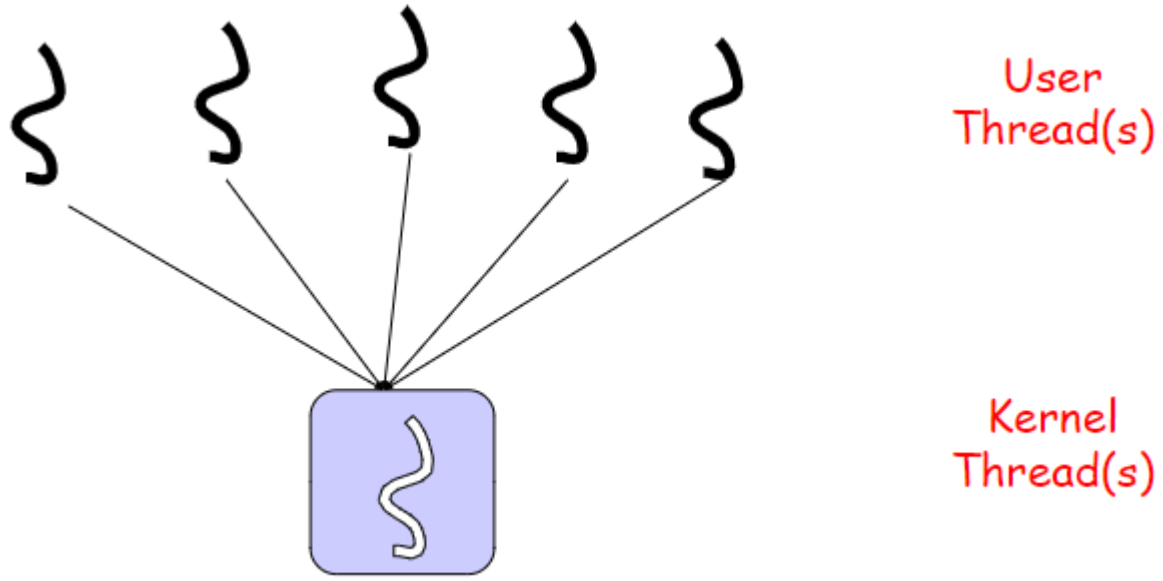
- **Maggiore overhead**: La gestione dei thread è più costosa e richiede più risorse, poiché il kernel deve intervenire per ogni operazione.
- **Maggiore complessità**: La gestione dei kernel thread richiede un'architettura più complessa all'interno del kernel.

Modelli di multithreading

Molti sistemi supportano sia kernel thread che user thread
Si vengono così a creare tre differenti modelli di multithreading:
→**Many-to-One**
→**One-to-One**
→**Many-to-Many**

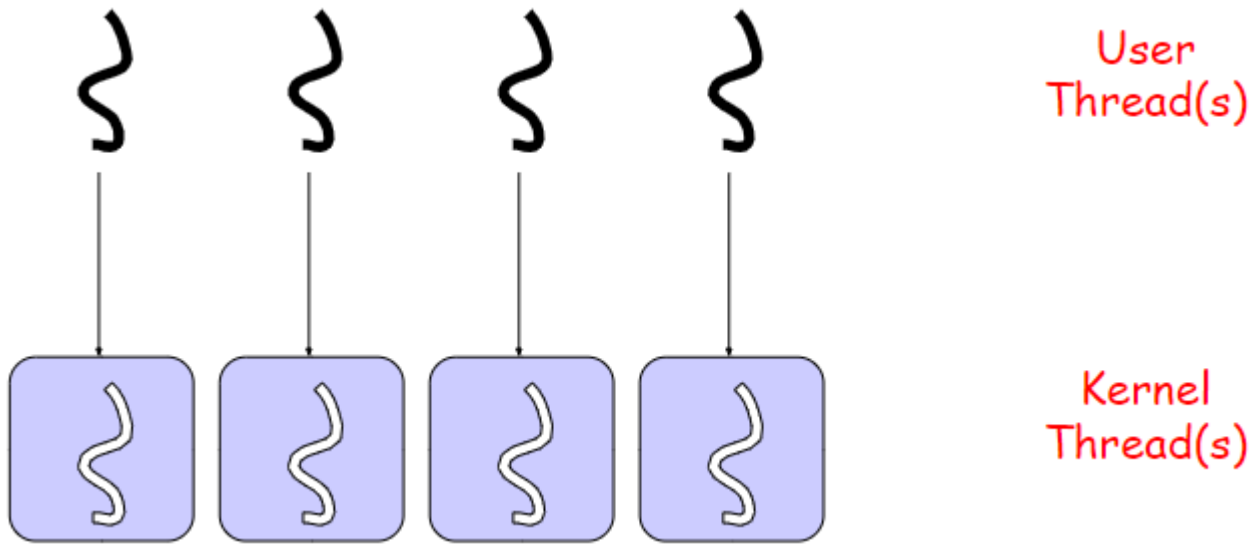
MANY TO ONE MULTITHREADING

Un certo numero di user thread vengono mappati su un solo kernel thread.
→Modello generalmente adottato da SO che non supportano kernel thread multipli



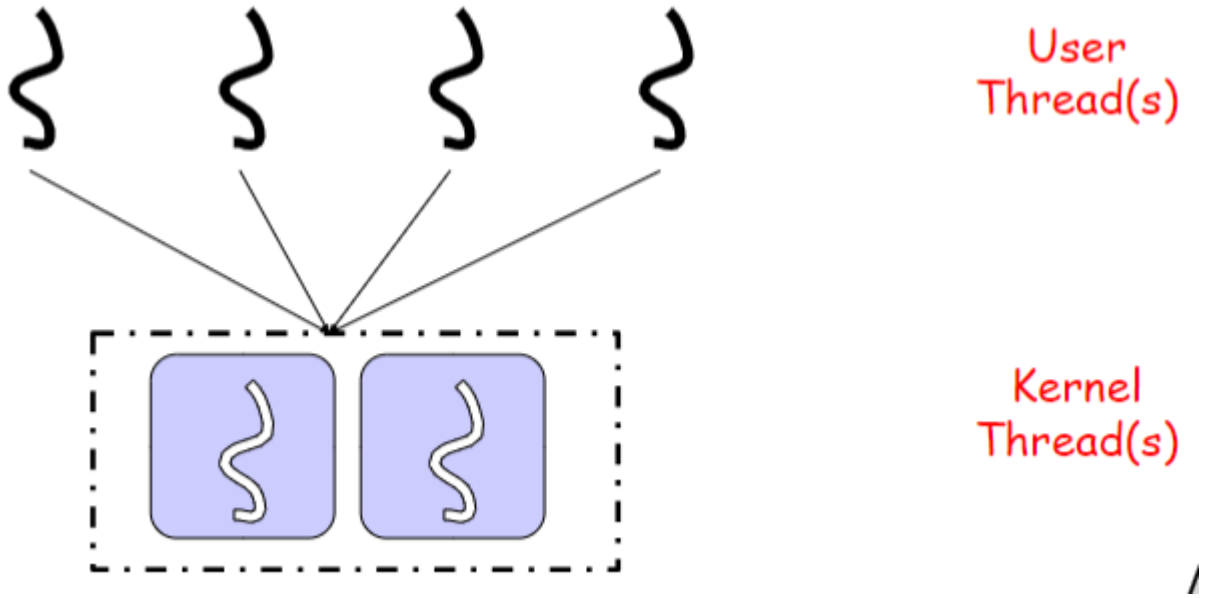
ONE TO ONE MULTITHREADING

Ogni user thread viene mappato su un kernel thread
→Può creare problemi di scalabilità per il kernel



MANY TO MANY MULTITHREADING

Riassume i benefici di entrambe le architetture →Supportato da Solaris, IRIX, Digital Unix



---

1. Many-to-One

Nel modello **Many-to-One**, **più user thread** (thread a livello utente) vengono mappati su un **singolo kernel thread**. In questo schema, i thread a livello utente vengono gestiti interamente in spazio utente senza l'intervento diretto del kernel.

Caratteristiche principali:

- **Un solo kernel thread**: L'intero gruppo di user thread dipende da un singolo kernel thread, quindi il kernel vede il processo come un'unica unità di esecuzione.

- **Gestione efficiente**: Poiché la gestione dei thread avviene solo nello spazio utente, le operazioni di creazione, distruzione e sincronizzazione dei thread sono rapide.
- **Assenza di vero parallelismo**: Solo un thread per processo può essere eseguito in un dato momento, quindi anche su sistemi multi-core, i thread non possono essere eseguiti in parallelo.

Vantaggi:

- **Basso overhead**: Le operazioni sui thread non richiedono interventi del kernel, rendendo il modello rapido ed efficiente in termini di risorse.
- **Semplicità**: Non richiede che il kernel sia a conoscenza dei singoli thread utente.

Svantaggi:

- **Blocco dell'intero processo**: Se un user thread si blocca, l'intero processo è bloccato.
- **Nessun parallelismo su multi-core**: Non permette ai thread di sfruttare i core multipli del processore.

Esempi:

- Linguaggi o sistemi con runtime che non supportano il multithreading del kernel, come alcune implementazioni di Python.

2. One-to-One

Nel modello **One-to-One**, ogni **user thread** è mappato su un **kernel thread** dedicato. In questo caso, ogni volta che un user thread viene creato, viene creato anche un kernel thread corrispondente.

Caratteristiche principali:

- **Associazione diretta**: Ogni user thread ha un kernel thread associato, quindi il kernel gestisce ogni user thread singolarmente.
- **Parallelismo**: I thread possono essere eseguiti in parallelo su sistemi multi-core, poiché il kernel può programmare i diversi kernel thread su CPU diverse.
- **Scalabilità limitata**: Poiché ogni user thread richiede un kernel thread, creare troppi thread può portare a un eccessivo overhead e un sovraccarico del sistema.

Vantaggi:

- **Parallelismo effettivo**: Supporta l'esecuzione parallela, permettendo di sfruttare pienamente i sistemi multi-core.
- **Blocco isolato**: Il blocco di un singolo thread non influenza gli altri thread del processo.

Svantaggi:

- **Elevato overhead**: Creare un kernel thread per ogni user thread richiede risorse significative, poiché ogni thread ha bisogno di memoria e risorse di gestione del kernel.
- **Possibile limitazione**: Alcuni sistemi limitano il numero di kernel thread che un singolo processo può avere, limitando la scalabilità.

Esempi:

- La maggior parte dei sistemi basati su **POSIX Threads** (Pthreads) usa questo modello.

3. Many-to-Many

Il modello **Many-to-Many** rappresenta un approccio **ibrido** in cui **molti user thread** vengono mappati su **molti kernel thread**, permettendo maggiore flessibilità. In questo schema, il numero di kernel thread può essere minore, uguale o maggiore del numero di user thread, e il sistema operativo decide come mappare i thread utente su quelli del kernel.

Caratteristiche principali:

- **Alta flessibilità**: I thread a livello utente possono essere gestiti in modo indipendente, e il kernel può programmare più kernel thread contemporaneamente per supportare il parallelismo.
- **Scalabilità**: Poiché il numero di kernel thread è separato dal numero di user thread, si può avere un numero elevato di user thread senza creare un elevato overhead sul kernel.

Vantaggi:

- **Parallelismo**: Supporta il parallelismo reale su sistemi multi-core, mappando user thread su kernel thread disponibili.
- **Gestione efficiente delle risorse**: Riduce l'overhead rispetto al modello One-to-One, poiché non richiede un kernel thread per ogni user thread.

Svantaggi:

- **Complessità**: La gestione e la sincronizzazione tra user thread e kernel thread è più complessa, poiché il kernel deve ottimizzare il bilanciamento tra thread utente e kernel.

Esempi:

- Alcune implementazioni avanzate di thread in **Windows** e in **Solaris** supportano il modello Many-to-Many.

Riassunto dei Modelli di Multithreading

Modello	Relazione tra thread	Parallelismo	Overhead	Esempi comuni
Many-to-One	Molti user thread su un kernel thread	Nessun parallelismo	Basso	Alcuni runtime e linguaggi come Python (senza GIL)
One-to-One	Un user thread per un kernel thread	Parallelismo	Elevato	Pthreads su Linux
Many-to-Many	Molti user thread su molti kernel thread	Parallelismo efficiente	Moderato	Solaris, Windows (con thread di sistema)

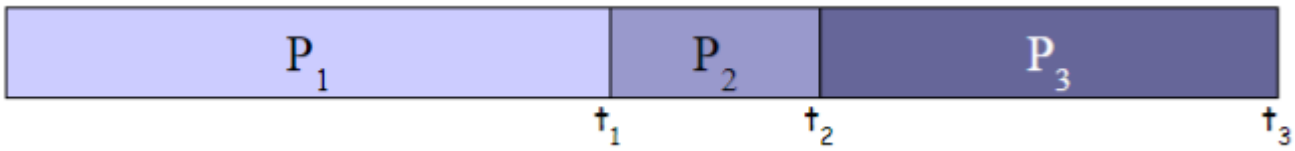
Ogni modello ha i suoi casi d'uso, e la scelta dipende dalle esigenze dell'applicazione e dalla capacità del sistema operativo di gestire in modo efficiente i thread.

SCHEDULING

Rappresentazione degli schedule

Diagramma di Gantt

per rappresentare uno schedule si usano i diagrammi di Gantt

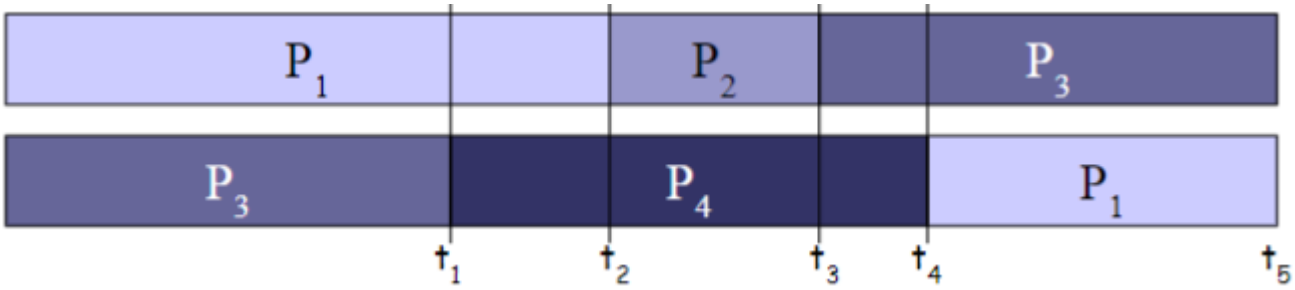


in questo esempio, la risorsa (es. CPU) viene utilizzata

- dal processo P1 dal tempo 0 a t1 ,
- viene quindi assegnata a P2 fino al tempo t2 e quindi
- a P3 fino al tempo t3

Diagramma di Gantt multi-risorsa

nel caso si debba rappresentare lo schedule di più risorse (per esempio un sistema multiprocessore) il diagramma di Gantt risulta composto da più righe parallele



Tipi di scheduler

Eventi che possono causare un context switch

1. quando un processo passa da stato **RUNNING** a stato **WAITING** (system call bloccante, operazione di I/O)
2. quando un processo passa dallo stato **RUNNING** allo stato **READY** (a causa di un interrupt)
3. quando un processo passa dallo stato **WAITING** allo stato **READY**
4. quando un processo termina

   Nota:
   nelle condizioni 1 e 4, l'unica scelta possibile è quella di selezionare un altro processo per l'esecuzione
   nelle condizioni 2 e 3, è possibile continuare ad eseguire il processo corrente.

==_ se

in altre parole: **il controllo della risorsa viene trasferito solo se l'assegnatario attuale lo cede volontariamente**
Windows 3.1, Mac OS y con y < 8

_ se

in altre parole:
tutti gli scheduler moderni

Vantaggi dello scheduling cooperativo: non richiede alcuni meccanismi hardware come ad esempio timer programmabili
Vantaggi dello scheduling preemptive: permette di utilizzare al meglio le risorse

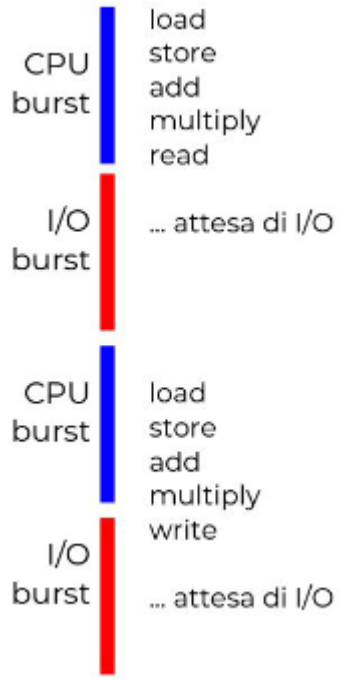
Criteri di scelta di uno scheduler

⇒***Utilizzo della risorsa (CPU)***
percentuale di tempo in cui la CPU è occupata ad eseguire processi
deve essere massimizzato
⇒***Throughput***
numero di processi completati per unità di tempo
dipende dalla lunghezza dei processi
deve essere massimizzato
⇒***Tempo di turnaround***

tempo che intercorre dalla sottomissione di un processo alla sua terminazione ⟶deve essere minimizzato

Tempo di attesa : il tempo trascorso da un processo nella coda ready ⟶deve essere minimizzato
Tempo di risposta : tempo che intercorre fra la sottomissione di un processo e il tempo di prima risposta
particolarmente significativo nei programmi interattivi, deve essere minimizzato

Caratteristiche dei processi
Durante l'esecuzione di un processo: ⇒si alternano periodi di attività svolte dalla CPU (CPU burst) e periodi di attività di I/O (I/O burst)
I processi:
⟶caratterizzati da CPU burst molto lunghi si dicono CPU bound
⟶caratterizzati da I/O burst molto lunghi si dicono I/O bound



>***Scheduling***
Algoritmi:
__First Come, First Served
Shortest-Job First
Shortest-Next-CPU-Burst First
Shortest-Remaining-Time-First
Round-Robin==__
⇒spiegazione

First Come, First Served (FCFS)

Algoritmo: il processo che arriva per primo, viene servito per primo ; politica senza preemption

Implementazione →tramite una coda (politica FIFO)

Problemi →elevati tempi medi di attesa e di turnaround → processi CPU bound ritardano i processi I/O bound

↳ sennuicemente lineare

Esempio: ordine di arrivo: P1 , P2 , P3

lunghezza dei CPU-burst in ms: 32, 2, 2

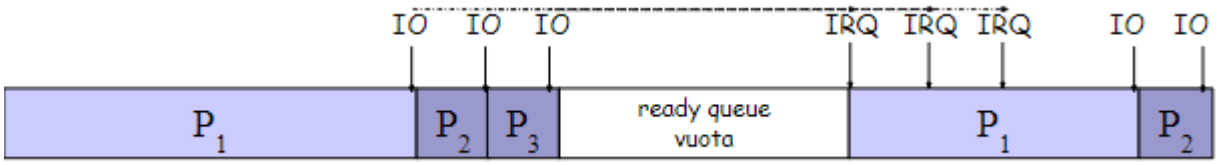
Tempo medio di turnaround (32+34+36)/3 = 34 ms

Tempo medio di attesa: (0+32+34)/3 = 22 ms

Supponiamo di avere :

- un processo CPU bound
- un certo numero di processi I/O bound
- i processi I/O bound si "mettono in coda" dietro al processo CPU bound, e in alcuni casi la ready queue si può svuotare

Convoy effect



Shortest Job First (SJF)

L'algoritmo SJF è ottimale rispetto al tempo di attesa, in quanto è possibile dimostrare che produce il minor tempo di attesa possibile ⇒ *ma è impossibile da implementare in pratica!*

è possibile solo fornire delle approssimazioni

Negli scheduler long-term possiamo chiedere a chi sottomette un job di predire la durata del job

Negli scheduler short-term non possiamo conoscere la lunghezza del prossimo CPU burst.... ma conosciamo la lunghezza di quelli precedenti

Calcolo approssimato della durata del CPU burst basata su media esponenziale dei CPU burst precedenti

→sia t(n) il tempo dell'n-esimo CPU burst e τ(n) la corrispondente previsione; τ(n+1) può essere calcolato come segue:

$$\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$$

Media esponenziale svolgendo la formula di ricorrenza, si ottiene

$$\tau_{n+1} = \Sigma_{j=0..n}\, \alpha(1-\alpha)^j t_{n-j} + (1-\alpha)^{n+1} \tau_o$$

da cui il nome media esponenziale

Spiegazione
t(n) rappresenta la storia recente
τ(n) rappresenta la storia passata
α rappresenta il peso relativo di storia passata e recente
cosa succede con α = 0, 1 oppure ½?
Nota importante: SJF può essere soggetto a starvation!

Shortest Job First "**APPROSSIMATO**" esiste in due versioni:
→ non preemptive il processo corrente esegue fino al completamento del suo CPU burst
→ preemptive il processo corrente può essere messo nella coda ready, se arriva un processo con un CPU burst più breve di quanto rimane da eseguire al processo corrente "Shortest-Remaining-Time First"

Round-Robin

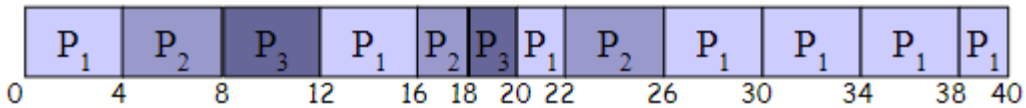
E' basato sul concetto di quanto di tempo (o time slice) un processo non può rimanere in esecuzione per un tempo superiore alla durata del quanto di tempo
Implementazione (1) : l'insieme dei processi pronti è organizzato come una coda
2 POSSIBILITA':
■ un processo può lasciare il processore volontariamente, in seguito ad un'operazione di I/O
■ un processo può esaurire il suo quanto di tempo senza completare il suo CPU burst, nel qual caso viene aggiunto in fondo alla coda dei processi pronti
in entrambi i casi, il prossimo processo da eseguire è il primo della coda dei processi pronti

⇒**La durata del quanto di tempo è un parametro critico del sistema**
se il quanto di tempo è breve, il sistema è meno efficiente perchè deve cambiare il processo attivo più spesso
se il quanto è lungo, in presenza di numerosi processi pronti ci sono lunghi periodi di inattività di ogni singolo processo, ⇒in sistemi interattivi, questo può essere fastidioso per gli utenti

Implementazione (2) : è necessario che l'hardware fornisca un timer (interval timer) che agisca come "sveglia" del processore -- il timer è un dispositivo che, attivato con un preciso valore di tempo, è in grado di fornire un interrupt allo scadere del tempo prefissa

Esempio

- tre processi P_1, P_2, P_3
- lunghezza dei CPU-burst in ms (P_1: 10+14; P_2: 6+4; P_3: 6)
- lunghezza del quanto di tempo: 4
- Tempo medio di turnaround (40+26+20)/3 = 28.66 ms
- Tempo medio di attesa: (16+16+14)/3 = 15.33 ms
  - NB (supponiamo attese di I/O brevi, < 2ms)
- Tempo medio di risposta: 4 ms



Primo ciclo (0 - 12 ms):

1. **Da 0 a 4 ms (P_1)**:
   - La CPU è assegnata a **P_1**, che esegue per **4 ms**.
   - Restano **10 - 4 = 6 ms** del primo burst.
   - **Prossimo processo: P_2.**
2. **Da 4 a 8 ms (P_2)**:
   - La CPU è assegnata a **P_2**, che esegue per **4 ms**.
   - Restano **6 - 4 = 2 ms** del primo burst.
   - **Prossimo processo: P_3.**
3. **Da 8 a 12 ms (P_3)**:
   - La CPU è assegnata a **P_3**, che esegue per **4 ms**.
   - Restano **6 - 4 = 2 ms**.
   - **Prossimo processo: P_1.**

Secondo ciclo (12 - 22 ms):

4. **Da 12 a 16 ms (P_1)**:
   - La CPU torna a **P_1**, che esegue per **4 ms**.
   - Restano **6 - 4 = 2 ms** del primo burst.
   - **Prossimo processo: P_2.**
5. **Da 16 a 18 ms (P_2)**:
   - La CPU torna a **P_2**, che esegue per **2 ms**, completando il suo primo burst.
   - **P_2 deve ancora eseguire il secondo burst di 4 ms**.
   - **Prossimo processo: P_3.**
6. **Da 18 a 20 ms (P_3)**:

- La CPU torna a **P_3**, che esegue per **2 ms**, completando il suo burst.
- **P_3 è rimosso dalla coda.**
- **Prossimo processo: P_1.**

Terzo ciclo (22 - 36 ms):

7. **Da 22 a 24 ms (P_1)**:
   - La CPU torna a **P_1**, che esegue per **2 ms**, completando il suo primo burst.
   - Passa al **secondo burst di 14 ms**.
   - **Prossimo processo: P_2.**
8. **Da 24 a 28 ms (P_2)**:
   - La CPU è assegnata a **P_2**, che esegue per **4 ms**, completando il suo secondo burst.
   - **P_2 è rimosso dalla coda.**
   - **Prossimo processo: P_1.**
9. **Da 28 a 32 ms (P_1)**:
   - La CPU torna a **P_1**, che esegue per **4 ms** dal secondo burst.
   - Restano **14 - 4 = 10 ms**.
   - **Prossimo processo: P_1.**
10. **Da 32 a 36 ms (P_1)**:
    - La CPU è assegnata a **P_1**, che esegue per **4 ms** dal secondo burst.
    - Restano **10 - 4 = 6 ms**.
    - **Prossimo processo: P_1.**

Quarto ciclo (36 - 40 ms):

11. **Da 36 a 40 ms (P_1)**:
    - La CPU è assegnata a **P_1**, che esegue per **4 ms** dal secondo burst.
    - Restano **6 - 4 = 2 ms**.

Ultima esecuzione (40 - 42 ms):

12. **Da 40 a 42 ms (P_1)**:
    - La CPU torna a **P_1**, che esegue per gli ultimi **2 ms**, completando il suo secondo burst.
    - **P_1 è rimosso dalla coda.**

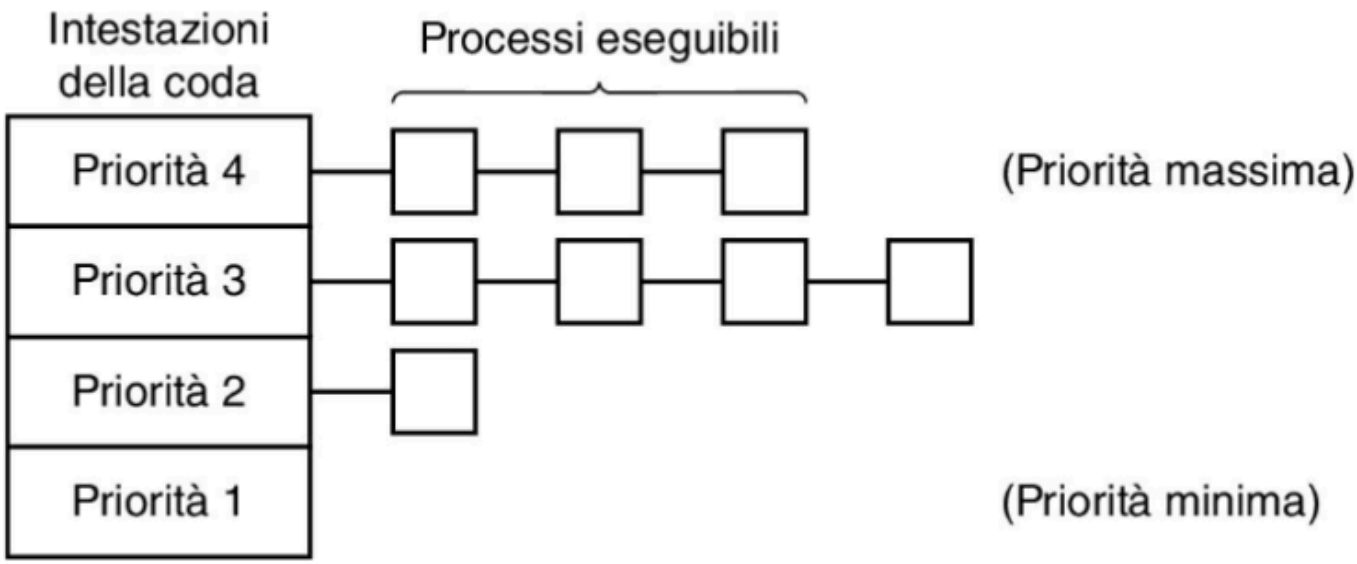
quindi il tempo medio di TOURN -AROUND è dato dalla somma dei "tempi" di termine di ogni singolo processo P, inclusi i doppi o tripli CPU burst.
mentre il tempo media di ATTESA è data dalla differenza di Tempo di fine del processo - lunghezza dei CPU burst, contando anche doppi e tripli eventuali

Scheduling a priorità

Round-robin : fornisce le stesse possibilità di esecuzione a tutti i processi
Ma i processi non sono tutti uguali: *usando round-robin puro la visualizzazione dei un video MPEG potrebbe essere ritardata da un processo che sta smistando la posta*
la lettera può aspettare ½ sec, il frame video NO

Descrizione a ogni processo è associata una specifica priorità
→lo scheduler sceglie il processo pronto con priorità più alta

Le priorità possono essere:
DEFINITE DAL SISTEMA OPERATIVO →vengono utilizzate una o più quantità misurabili per calcolare la priorità di un processo
esempio: SJF è un sistema basato su priorità
DEFINITE ESTERNAMENTE → le priorità non vengono definite dal sistema operativo, ma vengono imposte dal livello utente



Priorità statica la priorità non cambia durante la vita di un processo
→problema: processi a bassa priorità possono essere posti in starvation da processi ad alta priorità
Priorità dinamica la priorità può variare durante la vita di un processo
→è possibile utilizzare metodologie di priorità dinamica per evitare starvation
Priorità basata su aging tecnica che consiste nell'incrementare gradualmente la priorità dei processi in attesa.
→Posto che il range di variazione delle priorità sia limitato, nessun processo rimarrà in attesa per un tempo indefinito perché prima o poi raggiungerà la priorità massima.

Scheduling a classi di priorità

Descrizione è possibile creare diverse classi di processi con caratteristiche simili e assegnare ad ogni classe specifiche priorità
→la coda ready viene quindi scomposta in molteplici "sotto-code", una per ogni classe di processi
Algoritmo uno scheduler a classi di priorità seleziona il processo da eseguire fra quelli pronti della classe a priorità massima che contiene processi

Scheduling Multilivello

Descrizione all'interno di ogni classe di processi, è possibile utilizzare una politica specifica adatta alle caratteristiche della classe.
uno scheduler multilivello cerca prima la classe di priorità massima che ha almeno un processo ready.
sceglie poi il processo da porre in stato running coerentemente con la politica specifica della classe
-Esempio
Quattro classi di processi (priorità decrescente)
processi server (priorità statica)
processi utente interattivi (round-robin)
altri processi utente (FIFO)
il processo vuoto (FIFO banale)

Scheduling Real-Time

Scheduling Real-Time

In un sistema real-time la correttezza dell'esecuzione non dipende solamente dal valore del risultato, ma anche dall'istante temporale nel quale il risultato viene emesso

Hard real-time le deadline di esecuzione dei programmi non devono essere superate in nessun caso
sistemi di controllo nei velivoli, centrali nucleari o per la cura intensiva dei malati

Soft real-time
errori occasionali sono tollerabili
ricostruzione di segnali audio-video, transazioni interattive

Processi periodici sono periodici i processi che vengono riattivati con una cadenza regolare (periodo) *esempi*: controllo assetto dei velivoli, basato su rilevazione periodica dei parametri di volo

Processi aperiodici i processi che vengono scatenati da un evento sporadico, ad esempio l'allarme di un rilevatore di pericolo

Esempi di Scheduler RT: Rate Monotonic

Descrizione è una politica di scheduling, valida alle seguenti condizioni
→ogni processo periodico deve completare entro il suo periodo
→tutti i processi sono indipendenti
→la preemption avviene istantaneamente e senza overhead
viene assegnata staticamente una priorità a ogni processo.
processi con frequenza più alta (i.e. periodo più corto) hanno priorità più alta
ad ogni istante, viene eseguito il processo con priorità più alta (facendo preemption se necessario).